Spintly API Technical Knowledge Base

| Property | Value |

| Purpose | Comprehensive, production-quality technical onboarding guide and reference specification for backend engineers, API integrators, and QA engineers establishing Spintly Type 1 integrations. |

| Source | Verified Spintly Postman Collection (`type 1.postman_collection.json`) + Official Spintly Technical Documentation. |

| Intended Use | API integration development, verification testing, troubleshooting diagnostics, and RAG knowledge-base grounding. |

Table of Contents

1. Introduction

The Spintly Type 1 Integration API Knowledge Base outlines how to connect an organization's existing internal databases—such as an HRMS, ERP, or Visitor Management system—with the Spintly Cloud platform.

A Type 1 Integration automates user provisioning, access control assignments, and credential status synchronization in real time. Rather than administering users manually through a dashboard GUI, client applications interact directly with Spintly's REST APIs, maintaining a unified database source of truth.

Conceptual Use Case

Imagine a company (e.g. "Acme") with 1,000 employees. Without integration, the HR administrator has to manually register every new employee in the corporate HR database, and then log into the Spintly Admin Dashboard to create the user a second time and assign door permissions. Doing this for onboarding, updates, transfers, and offboarding is highly cumbersome and prone to database inconsistencies.

By integrating via Spintly Type 1 APIs:

1. Whenever a new user is created in the corporate HR portal, the portal backend automatically calls the Spintly Create User API.

2. The Spintly system registers the user and returns a unique identifier mapped back to the corporate DB.

3. Access permissions can be automatically assigned to default doors without human intervention.

Live Integration Example: FF21

This solution is implemented in production by FF21, a co-living spaces provider in Bangalore operating over 1,000 beds. FF21 uses Spintly Type 1 APIs to automate room key provisioning and access control permissions, allowing tenants to unlock authorized building doors instantly via their mobile application upon booking confirmation.

2. Authentication APIs

OAuth Token

Operation ID

`oauth_token`

Purpose

Authenticates the client application and generates a temporary Bearer token required for all subsequent data API calls.

Prerequisites

- Active Spintly client credentials (`<CLIENT_ID>` and `<CLIENT_SECRET>`).

- Network access to the AWS Cognito API Gateway.

HTTP Method

`POST`

Endpoint

`https://qlwtofmb58.execute-api.ap-south-1.amazonaws.com/prod/saamsIdm/oauth/token`

Authorization

Type: `None`

Request Headers

`Content-Type: application/json`

Request Body

json

```
{
  "clientId": "<CLIENT_ID>",
  "clientSecret": "<CLIENT_SECRET>",
  "grantType": "urn:ietf:params:oauth:grant-type:client-credentials"
}
```

Field Explanation

| Field | Type | Required | Description |

| `clientId` | String | Yes | Unique application identifier provided by Spintly. |

| `clientSecret` | String | Yes | Confidential security credential corresponding to the client ID. |

| `grantType` | String | Yes | OAuth 2.0 flow type. Must be set to `"urn:ietf:params:oauth:grant-type:client-credentials"`. |

Postman Verification Steps

1. Open Postman.

2. Create a new request. Set the HTTP method to **POST**.

3. Enter the endpoint: `https://qlwtofmb58.execute-api.ap-south-1.amazonaws.com/prod/saamsIdm/oauth/token`

4. Under the **Headers** tab, add `Content-Type: application/json`.

5. Under the **Authorization** tab, set the type to **No Auth**.

6. Under the **Body** tab, select **raw** and set the format to **JSON**.

7. Enter the Request Body JSON replacing the credentials placeholders.

8. Click **Send** to dispatch the HTTP request.

9. Record the returned `access_token` from the response.

Success Response

json


```
{
  "type": "success",
  "message": {
    "payload": {
      "access_token": "<ACCESS_TOKEN>",
      "refresh_token": "<REFRESH_TOKEN>",
      "token_type": "bearer",
      "expires_in": 7776000,
      "issued_token_type": "urn:ietf:params:oauth:token-type:jwt"
    },
    "statusCode": 200
  }
```


3. Organisation APIs

Get Sites

Operation ID

`get_sites`

Purpose

Retrieves a paginated list of physical sites (locations, facilities) configured within the organization.

Prerequisites

- OAuth authentication completed.

- Bearer token acquired.

HTTP Method

`POST`

Endpoint

`https://saams.api.spintly.com/organisationManagement/v2/integrator/organisations/{organisationId}/sites`

Authorization

Type: `Bearer <ACCESS_TOKEN>`

Request Headers

`Authorization: Bearer <ACCESS_TOKEN>`

`Content-Type: application/json`

Path Parameters

| Parameter | Description |

| `organisationId` | The unique identifier of the organization tenant. |

Request Body

json

```
{
  "pagination": {
    "page": 1,
    "perPage": 40
  }
}
```

Field Explanation

| Field | Type | Required | Description |

| `pagination.page` | Integer | Yes | The target page number to retrieve. |

| `pagination.perPage` | Integer | Yes | The max number of sites returned per page. |

Postman Verification Steps

1. Open Postman.

2. Create a new request. Set the HTTP method to **POST.**

3. Enter the endpoint: `https://saams.api.spintly.com/organisationManagement/v2/integrator/organisations/{organisationId}/sites` (replace `{organisationId}`).

4. Under the **Headers** tab, add `Authorization: Bearer <ACCESS_TOKEN>` and `Content-Type: application/json`.

5. Under the **Body** tab, select **raw** and set the format to **JSON**.

6. Paste the pagination JSON structure.

7. Click **Send**.

Success Response

json


```
{
  "type": "success",
  "message": {
    "sites": [
      {
        "id": 13480,
        "name": "Site A",
        "location": "Goa",
        "createdAt": "2026-06-25T10:36:24.418799Z",
        "updatedAt": "2026-06-25T10:36:27.066736Z"
      }
    ],
    "pagination": {
      "currentPage": 1,
      "perPage": 40,
      "total": 1
    }
```

}

}

Get Roles

Operation ID

`get_roles`

Purpose

Retrieves all roles configured in the organization. These roles define access categories (such as Employee, Contractor, Visitor) which must be assigned to users.

Prerequisites

- OAuth authentication completed.

- Bearer token acquired.

HTTP Method

`GET`

Endpoint

`https://saams.api.spintly.com/userManagement/integrator/v1/organisations/{organisationId}/formData?roles=roles`

Authorization

-Type: `Bearer <ACCESS_TOKEN>`

Request Headers

- `Authorization: Bearer <ACCESS_TOKEN>`

- `Content-Type: application/json`

Path Parameters

| Parameter | Description |

| `organisationId` | The unique identifier of the organization tenant. |

Query Parameters

| Parameter | Required | Description |

| `roles` | Yes | Filters form data type. Must be set to `roles`. |

Request Body

None

Postman Verification Steps

1. Open Postman.

2. Create a new request. Set the HTTP method to **GET**.

3. Enter the URL: `https://saams.api.spintly.com/userManagement/integrator/v1/organisations/{organisationId}/formData?roles=roles` (replace `{organisationId}`).

4. Under the **Headers** tab, add `Authorization: Bearer <ACCESS_TOKEN>` and `Content-Type: application/json`.

5. Click **Send**.

Success Response

json


```
{
  "type": "success",
  "message": {
    "roles": [
      {
        "id": 24304,
        "name": "super_admin"
      },
```

```
    {
      "id": 24305,
      "name": "site_admin"
    },
    {
      "id": 24306,
      "name": "end_user"
    }
  ]
 }
}
```


4. User Management APIs

Create User

Operation ID

`create_user`

Purpose

Registers a new user record in the organization. The `homeSiteId` and `roles` array must reference pre-existing IDs retrieved from configuration APIs.

Prerequisites

- Site IDs and Role IDs cached.

- Active OAuth token.

HTTP Method

`POST`

Endpoint

`https://saams.api.spintly.com/userManagement/integrator/v1/organisations/{organisationId}/users/create`

Authorization

- Type: `Bearer <ACCESS_TOKEN>`

Request Headers

- `Authorization: Bearer <ACCESS_TOKEN>`
- `Content-Type: application/json`

Path Parameters

Parameter	Description
`organisationId`	The unique identifier of the organization tenant.

Request Body

json


```
{
  "name": "j1ane doe",
  "phone": "",
  "roles": [24306],
  "adminOfSites": [null],
  "devicelock": true,
  "accessData": {
    "mobile": true,
    "card": true,
    "fingerprint": true,
    "remoteAccess": false,
    "clickToAccessRange": 1,
    "accessExpiresAt": null,
    "tapToAccess": true,
```

```
    "accessPoints": [],
    "gps": true,
    "deviceLock": true,
    "faceAccess": true
  },
  "homeSiteId": 13480,
  "terms": [],
  "employeeCode": "10111",
  "reportingTo": null,
  "probationPeriod": null,
  "joiningDate": null,
  "credentialId": null
}
```


Field Explanation

| Field | Type | Required | Description |

| `name` | String | Yes | The user's full name. |

| `phone` | String | Yes | Mobile contact number. |

| `roles` | Array of Integers | Yes | List of role IDs to associate with this user. |

| `devicelock` | Boolean | Yes | Restricts app execution to a single physical device profile. |

| `accessData` | Object | Yes | Access configuration permissions (Bluetooth mobile, fingerprint, cards). |

| `homeSiteId` | Integer | Yes | The ID of the user's primary assigned site location. |

| `employeeCode` | String | Yes | Organizational identifier code. |

Postman Verification Steps

1. Open Postman.

2. Create a new request. Set the HTTP method to **POST**.

3. Enter the endpoint: `https://saams.api.spintly.com/userManagement/integrator/v1/organisations/{organisationId}/users/create` (replace `{organisationId}`).

4. Under the **Headers** tab, add `Authorization: Bearer <ACCESS_TOKEN>` and `Content-Type: application/json`.

5. Under the **Body** tab, select **raw** and set the format to **JSON**.

6. Enter the JSON payload containing the user parameters.

7. Click **Send**.

8. Record the returned user identifier `userId` from the success response body.

Success Response

json


```
{
  "type": "success",
  "message": {
    "displayMessage": "You have successfully added the user. You can also edit the details later on from the edit details page in the User details.",
    "userName": "j1ane doe",
    "showName": true,
    "orgUserId": "<ORG_USER_ID>",
    "userId": "<USER_ID>"
  }
}
```


Update User

Operation ID

`update_user`

Purpose

Updates attributes (name, role, site, employee code) for one or more existing users in a bulk operation.

Prerequisites

- Active OAuth token.

- Existing user ID.

HTTP Method

`PATCH`

Endpoint

`https://saams.api.spintly.com/userManagement/integrator/v1/organisations/{organisationId}/users/update`

Authorization

- Type: `Bearer <ACCESS_TOKEN>`

Request Headers

- `Authorization: Bearer <ACCESS_TOKEN>`

- `Content-Type: application/json`

Path Parameters

| Parameter | Description |

| `organisationId` | The unique identifier of the organization tenant. |

Request Body

json

```
{
  "users": [
    {
      "id": 1197835,
      "accessExpiresAt": null,
```

```
    "employeeCode": "101",
    "name": "jane doe11",
    "reportingTo": null,
    "homeSiteId": 13480,
    "roles": [
      24306
    ],
    "terms": [],
    "joiningDate": null,
    "attributes": []
   }
 ]
}
```

Field Explanation

| Field | Type | Required | Description |

| `users` | Array | Yes | Bulk array containing modified user payload structures. |

| `users[].id` | Integer | Yes | Unique Spintly user identifier to update. |

| `users[].homeSiteId` | Integer | Yes | Modified site ID assignment. |

Postman Verification Steps

1. Open Postman.

2. Create a new request. Set the HTTP method to **PATCH**.

3. Enter the endpoint: `https://saams.api.spintly.com/userManagement/integrator/v1/organisations/{organisationId}/users/update` (replace `{organisationId}`).

4. Under the **Headers** tab, add `Authorization: Bearer <ACCESS_TOKEN>` and `Content-Type: application/json`.

5. Under the **Body** tab, select **raw** and set the format to **JSON**.

6. Paste the request body JSON, replacing placeholders with your target updates.

7. Click **Send**.

Success Response

json

```
{
  "type": "success",
  "message": "success"
}
```

Deactivate User

Operation ID

`deactivate_user`

Purpose

Disables access credentials for a user in Spintly without deleting their profile records.

Prerequisites

- Active OAuth token.
- Existing user ID.

HTTP Method

`PATCH`

Endpoint

`https://saams.api.spintly.com/userManagement/integrator/v1/organisations/{organisationId}/users/update`

Authorization

- Type: `Bearer <ACCESS_TOKEN>`

Request Headers

- `Authorization: Bearer <ACCESS_TOKEN>`
- `Content-Type: application/json`

Path Parameters

| Parameter | Description |

| `organisationId` | The unique identifier of the organization tenant. |

Request Body

json

```
{
  "users": [
    {
      "id": 1197835,
      "deactivateUser": true
    }
  ]
}
```

Field Explanation

| Field | Type | Required | Description |

`users`	Array	Yes	Bulk array containing user update parameters.
`users[].id`	Integer	Yes	Unique Spintly user identifier to update.
`users[].deactivateUser`	Boolean	Yes	Flag indicating user deactivation. Must be set to `true`.

Postman Verification Steps

1. Open Postman.
2. Create a new request. Set the HTTP method to **PATCH**.

3. Enter the endpoint: `https://saams.api.spintly.com/userManagement/integrator/v1/organisations/{organisationId}/users/update` (replace `{organisationId}`).

4. Under the **Headers** tab, add `Authorization: Bearer <ACCESS_TOKEN>` and `Content-Type: application/json`.

5. Under the **Body** tab, select **raw** and set the format to **JSON**.

6. In the raw JSON body, set "deactivateUser": true for the target user `id`.

7. Click **Send**.

Success Response

json

```
{
  "type": "success",
  "message": "success"
}
```

Activate User

Operation ID

`activate_user`

Purpose

Re-enables access credentials for a previously deactivated user record.

Prerequisites

- Active OAuth token.

- Existing user ID.

HTTP Method

`PATCH`

Endpoint

`https://saams.api.spintly.com/userManagement/integrator/v1/organisations/{organisationId}/users/update`

Authorization

- Type: `Bearer <ACCESS_TOKEN>`

Request Headers

- `Authorization: Bearer <ACCESS_TOKEN>`
- `Content-Type: application/json`

Path Parameters

Parameter	Description
`organisationId`	The unique identifier of the organization tenant.

Request Body

json


```
{
  "users": [
    {
      "id": 1197835,
      "deactivateUser": false,
      "accessExpiresAt": null,
      "employeeCode": "",
      "name": "john",
      "reportingTo": "",
      "homeSiteId": 13480,
      "adminOfSites": [],
      "roles": [
        24306
      ],
      "terms": [],
```

```
    "accessPoints": [],
    "joiningDate": "2024-04-05",
    "attributes": [],
    "accessData": {
      "gps": false,
      "accessExpiresAt": null,
      "accessPoints": [],
      "mobile": true
    }
  }
 ]
}
```


Field Explanation

| Field | Type | Required | Description |

| `users` | Array | Yes | Bulk array containing user update parameters. |

| `users[].id` | Integer | Yes | Unique Spintly user identifier to update. |

| `users[].deactivateUser` | Boolean | Yes | Flag indicating user deactivation. Must be set to `false` for activation. |

Postman Verification Steps

1. Open Postman.

2. Create a new request. Set the HTTP method to **PATCH**.

3. Enter the endpoint: `https://saams.api.spintly.com/userManagement/integrator/v1/organisations/{organisationId}/users/update` (replace `{organisationId}`).

4. Under the **Headers** tab, add `Authorization: Bearer <ACCESS_TOKEN>` and `Content-Type: application/json`.

5. Under the **Body** tab, select **raw** and set the format to **JSON**.

6. In the raw JSON body, set `"deactivateUser": false` along with the other required user attributes from the documentation context.

7. Click **Send**.

Success Response

json

{

"type": "success",

"message": {

"displayMessage": "You have successfully activated the user . You can also edit the details later on from the edit details page in the User details.",

"showName": true,

"userName": "john"

}

}

Delete User

Operation ID

`delete_user`

Purpose

Permanently deletes user accounts from the organization.

Prerequisites

- Active OAuth token.

- Existing user ID.

HTTP Method

`POST`

Endpoint

`https://saams.api.spintly.com/userManagement/integrator/v1/organisations/{organisationId}/users/delete`

Authorization

- Type: `Bearer <ACCESS_TOKEN>`

Request Headers

- `Authorization: Bearer <ACCESS_TOKEN>`
- `Content-Type: application/json`

Path Parameters

| Parameter | Description |

| `organisationId` | The unique identifier of the organization tenant. |

Request Body

json

```
{
  "userIds": [
    1197835
  ]
}
```

Field Explanation

| Field | Type | Required | Description |

| `userIds` | Array of Integers | Yes | List of unique Spintly user IDs to permanently delete. |

Postman Verification Steps

1. Open Postman.
2. Create a new request. Set the HTTP method to **POST**.
3. Enter the endpoint: `https://saams.api.spintly.com/userManagement/integrator/v1/organisations/{organisationId}/users/delete` (replace `{organisationId}`).

4. Under the **Headers** tab, add `Authorization: Bearer <ACCESS_TOKEN>` and `Content-Type: application/json`.

5. Under the **Body** tab, select **raw** and set the format to **JSON**.

6. Input the `userIds` array in the JSON request body containing the target ID.

7. Click **Send**.

Success Response

json

```
{
  "type": "success",
  "message": "success"
}
```

Fetch All Organisation Users

Operation ID

`fetch_all_organisation_users`

Purpose

Lists all registered users within the organization, supporting pagination and status filtering.

Prerequisites

- Active OAuth token.

HTTP Method

`POST`

Endpoint

`https://saams.api.spintly.com/userManagement/integrator/v1/organisations/{organisationId}/users`

Authorization

- Type: `Bearer <ACCESS_TOKEN>`

Request Headers

- `Authorization: Bearer <ACCESS_TOKEN>`
- `Content-Type: application/json`

Path Parameters

| Parameter | Description |

| `organisationId` | The unique identifier of the organization tenant. |

Request Body

json


```
{
  "pagination": {
    "page": 1,
    "perPage": 25,
    "currentPage": 1
  },
  "filters": {
    "createdOn": null,
    "userType": [
      "active"
    ],
    "accessExpiresAt": null,
    "s": {
      "email": ""
    },
    "terms": [],
    "sites": []
```

```
  }
}
```

Field Explanation

| Field | Type | Required | Description |

| `pagination.page` | Integer | Yes | The target page number. |

| `pagination.perPage` | Integer | Yes | The maximum number of users returned per page. |

| `filters.userType` | Array of Strings | Yes | User status filters (e.g. `"active"`). |

Postman Verification Steps

1. Open Postman.

2. Create a new request. Set the HTTP method to **POST**.

3. Enter the URL: `https://saams.api.spintly.com/userManagement/integrator/v1/organisations/{organisationId}/users` (replace `{organisationId}`).

4. Under the **Headers** tab, add `Authorization: Bearer <ACCESS_TOKEN>` and `Content-Type: application/json`.

5. Under the **Body** tab, select **raw** and set the format to **JSON**.

6. Enter the pagination and filter details.

7. Click **Send**.

Success Response

json

```
{
  "type": "success",
  "message": {
    "pagination": {
      "total": 8,
      "perPage": 25,
      "currentPage": 1
    },
```

```
"users": [
 {
  "id": "<USER_ID>",
  "name": " <USER_NAME>",
  "email": " <EMAIL>",
  "phone": " <PHONE_NUMBER>",
  "createdAt": "2026-06-25T10:36:29.003058Z",
  "roles": [
   {
    "id": 24304,
    "name": "super_admin"
   },
   {
    "id": 24306,
    "name": "end_user"
   }
  ],
  "employeeCode": "",
  "homeSite": {
   "Id": 13480,
   "name": "Site A"
  },
  "isSignedUp": false,
  "poolSub": "",
  "deactivatedOn": null,
  "profilePicture": {
   "image": null
  },
  "uniqueId": null,
  "state": "create_pending"
 }
```

```
    ]
  }
}
```

5. Access Point APIs

Get Access Points

Operation ID

`get_access_points`

Purpose

Retrieves all physical access barriers (doors, turnstiles, readers) that are managed inside the organization.

Prerequisites

- OAuth authentication completed.

- Bearer token acquired.

HTTP Method

`GET`

Endpoint

`https://saams.api.spintly.com/accessManagementV3/integrator/v1/organisations/{organisationId}/getAccessPointList`

Authorization

- Type: `Bearer <ACCESS_TOKEN>`

Request Headers

- `Authorization: Bearer <ACCESS_TOKEN>`

- `Content-Type: application/json`

Path Parameters

| Parameter | Description |

| `organisationId` | The unique identifier of the organization tenant. |

Request Body

None

Postman Verification Steps

1. Open Postman.

2. Create a new request. Set the HTTP method to **GET**

3. Enter the URL: `https://saams.api.spintly.com/accessManagementV3/integrator/v1/organisations/{organisationId}/getAccessPointList` (replace `{organisationId}`).

4. Under the **Headers** tab, add `Authorization: Bearer <ACCESS_TOKEN>` and `Content-Type: application/json`.

5. Click **Send**.

Success Response

json

```
{
  "type": "success",
  "message": {
    "accessPoints": []
  }
}
```

6. Permission Management APIs

Get User Permissions

Operation ID

`get_user_permissions`

Purpose

Retrieves access permissions currently assigned to a user. Specifying site IDs in the `sites` array filters the query scope.

Prerequisites

- Active OAuth token.

- Existing user ID.

HTTP Method

`POST`

Endpoint

`https://saams.api.spintly.com/accessManagementV3/integrator/v1/organisations/{organisationId}/users/{userId}/permissions`

Authorization

- Type: `Bearer <ACCESS_TOKEN>`

Request Headers

- `Authorization: Bearer <ACCESS_TOKEN>`

- `Content-Type: application/json`

Path Parameters

| Parameter | Description |

| `organisationId` | The unique identifier of the organization tenant. |

| `userId` | The unique identifier of the target user. |

Request Body

json

```
{
  "sites": []
}
```

Field Explanation

| Field | Type | Required | Description |

| `sites` | Array | Yes | List of Site IDs to filter the scope. Pass an empty array `[]` to query all sites. |

Postman Verification Steps

1. Open Postman.

2. Create a new request. Set the HTTP method to **POST**.

3. Enter the URL: `https://saams.api.spintly.com/accessManagementV3/integrator/v1/organisations/{organisationId}/users/{userId}/permissions` (replace placeholders).

4. Under the **Headers** tab, add `Authorization: Bearer <ACCESS_TOKEN>` and `Content-Type: application/json`.

5. Under the **Body** tab, select **raw** and set the format to **JSON**.

6. Enter the request body JSON (an empty `"sites": [] array retrieves permissions across all sites).

7. Click **Send**.

Success Response

json


```
{
  "type": "success",
  "message": {
    "permissionAssigned": [],
    "permissionNotAssigned": [],
    "pendingPermissions": []
```

```
  }
}
```

Grant Access Point User Permissions

Operation ID

`grant_access_point_permissions`

Purpose

Grants access to a specific access point (e.g. reader/door) for a list of users.

Prerequisites

- Active OAuth token.
- Target user ID and access point ID.

HTTP Method

`PATCH`

Endpoint

`https://saams.api.spintly.com/accessManagementV3/integrator/v1/organisations/{organisationId}/accessPoint/{accessPointId}/users/permissions`

Authorization

- Type: `Bearer <ACCESS_TOKEN>`

Request Headers

- `Authorization: Bearer <ACCESS_TOKEN>`
- `Content-Type: application/json`

Path Parameters

| Parameter | Description |

| `organisationId` | The unique identifier of the organization tenant. |

| `accessPointId` | The unique identifier of the access point. |

Request Body

json

```
{
  "permissionsToAdd": [
    1197306
  ],
  "permissionsToRemove": [],
  "pendingPermissionsToRemove": []
}
```

Field Explanation

| Field | Type | Required | Description |

| `permissionsToAdd` | Array of Integers | Yes | User IDs to grant permissions for this Access Point. |

| `permissionsToRemove` | Array of Integers | Yes | User IDs to revoke permissions for this Access Point. |

| `pendingPermissionsToRemove` | Array | Yes | Pending user permissions array to dismiss. |

Postman Verification Steps

1. Open Postman.

2. Create a new request. Set the HTTP method to **PATCH**.

3. Enter the endpoint: `https://saams.api.spintly.com/accessManagementV3/integrator/v1/organisations/{organisationId}/accessPoint/{accessPointId}/users/permissions` (replace placeholders).

4. Under the **Headers** tab, add `Authorization: Bearer <ACCESS_TOKEN>` and `Content-Type: application/json`.

5. Under the **Body** tab, select **raw** and set the format to **JSON**.

6. Define the user IDs to authorize inside the `permissionsToAdd` array.

7. Click **Send**.

Grant User Permissions

Operation ID

`grant_user_permissions`

Purpose

The Postman request named **"Grant User Permissions"** uses the access-point permissions endpoint with the target user ID in permissionsToRemove, which removes the user's access permission from the access point.

Prerequisites

- Active OAuth token.

- Target user ID and access point ID.

HTTP Method

`PATCH`

Endpoint

`https://saams.api.spintly.com/accessManagementV3/integrator/v1/organisations/{organisationId}/accessPoint/{accessPointId}/users/permissions`

Authorization

- Type: `Bearer <ACCESS_TOKEN>`

Request Headers

- `Authorization: Bearer <ACCESS_TOKEN>`

- `Content-Type: application/json`

Path Parameters

| Parameter | Description |

| `organisationId` | The unique identifier of the organization tenant. |

| `accessPointId` | The unique identifier of the access point. |

Query Parameters

None

Request Body

json

```
{
  "permissionsToAdd": [],
  "permissionsToRemove": [
    1197306
  ],
  "pendingPermissionsToRemove": []
}
```

Field Explanation

| Field | Type | Required | Description |

| `permissionsToAdd` | Array of Integers | Yes | User IDs to grant permissions for this Access Point. |

| `permissionsToRemove` | Array of Integers | Yes | User IDs to revoke permissions for this Access Point. |

| `pendingPermissionsToRemove` | Array | Yes | Pending user permissions array to dismiss. |

Postman Verification Steps

1. Open Postman.

2. Create a new request. Set the HTTP method to **PATCH**.

3. Enter the endpoint: `https://saams.api.spintly.com/accessManagementV3/integrator/v1/organisations/{organisationId}/accessPoint/{accessPointId}/users/permissions` (replace placeholders).

4. Under the **Headers** tab, add `Authorization: Bearer <ACCESS_TOKEN>` and `Content-Type: application/json`.

5. Under the **Body** tab, select **raw** and set the format to **JSON**.

6. Enter the JSON payload setting the target user ID inside the `permissionsToRemove` array.

7. Click **Send**.

7. Meeting APIs

Create Meeting

Operation ID

`create_meeting`

Purpose

Registers a scheduled visitor meeting, assigning temporary access permissions to a visitor.

Prerequisites

- Active OAuth token.

HTTP Method

`POST`

Endpoint

`https://saams.api.spintly.com/visitorManagementV3/integrator/v1/organisations/{organisationId}/meeting`

Authorization

- Type: `Bearer <ACCESS_TOKEN>`

Request Headers

- `Authorization: Bearer <ACCESS_TOKEN>`
- `Content-Type: application/json`

Path Parameters

Parameter	Description
`organisationId`	The unique identifier of the organization tenant.

Request Body

json

```
{
  "userId": "1190834",
  "startTime": 1781883600,
  "endTime": 1781942400,
  "permissionsToAdd": [
    23242
  ]
}
```

Field Explanation

Field	Type	Required	Description
`userId`	String	Yes	Identifier of the host employee.
`startTime`	Long (Epoch)	Yes	Meeting start time represented as a Unix Epoch timestamp.
`endTime`	Long (Epoch)	Yes	Meeting end time represented as a Unix Epoch timestamp.
`permissionsToAdd`	Array of Integers	Yes	List of Access Point IDs to grant access to for the meeting.

Postman Verification Steps

1. Open Postman.

2. Create a new request. Set the HTTP method to **POST**

3. Enter the endpoint: `https://saams.api.spintly.com/visitorManagementV3/integrator/v1/organisations/{organisationId}/meeting` (replace `{organisationId}`).

4. Under the **Headers** tab, add `Authorization: Bearer <ACCESS_TOKEN>` and `Content-Type: application/json`.

5. Under the **Body** tab, select **raw** and set the format to **JSON**

6. Input the request body JSON, specifying the host user ID, timing epoch values, and access point IDs inside the `permissionsToAdd` array.

7. Click **Send**.

8. Record the returned `meetingId` and `visitorId` from the success response body.

Success Response

json


```
{
  "type": "success",
  "message": {
    "meetingId": 86600,
    "visitorId": 51194,
    "qrData":
"02f3b6ea3cd062356a8048366a9e5c886a0000a341584b37843dcca1fb1b64253bb5faf123bb206a0cb9280691e6e433af6a15",
    "credentialId": 1022015219
  }
}
```


Update Meeting

Operation ID

`update_meeting`

Purpose

Modifies meeting parameters or updates visitor access permissions for an active meeting ID.

Prerequisites

- Active OAuth token.

- Existing meeting ID.

HTTP Method

`PATCH`

Endpoint

`https://saams.api.spintly.com/visitorManagementV3/integrator/v1/organisations/{organisationId}/meeting/{meetingId}/update`

Authorization

- Type: `Bearer <ACCESS_TOKEN>`

Request Headers

- `Authorization: Bearer <ACCESS_TOKEN>`

- `Content-Type: application/json`

Path Parameters

| Parameter | Description |

| `organisationId` | The unique identifier of the organization tenant. |

| `meetingId` | The unique identifier of the active meeting. |

Request Body

json

```
{
  "startTime": 1774009800,
  "endTime": 1774012500,
  "permissionsToAdd": [
    23242
  ],
  "permissionsToRemove": []
}
```

Field Explanation

| Field | Type | Required | Description |

| `startTime` | Long (Epoch) | Yes | Meeting start time represented as a Unix Epoch timestamp. |

| `endTime` | Long (Epoch) | Yes | Meeting end time represented as a Unix Epoch timestamp. |

| `permissionsToAdd` | Array of Integers | Yes | Access Point IDs to grant access to. |

| `permissionsToRemove` | Array of Integers | Yes | Access Point IDs to revoke access from. |

Postman Verification Steps

1. Open Postman.

2. Create a new request. Set the HTTP method to **PATCH**

3. Enter the URL: `https://saams.api.spintly.com/visitorManagementV3/integrator/v1/organisations/{organisationId}/meeting/{meetingId}/update` (replace placeholders).

4. Under the **Headers** tab, add `Authorization: Bearer <ACCESS_TOKEN>` and `Content-Type: application/json`.

5. Under the **Body** tab, select **raw** and set the format to **JSON**.

6. Input request body modifications.

7. Click **Send**.

Delete Meeting

Operation ID

`delete_meeting`

Purpose

Cancels a scheduled meeting and immediately revokes all associated access permissions.

Prerequisites

- Active OAuth token.

- Existing meeting ID.

HTTP Method

`DELETE`

Endpoint

`https://saams.api.spintly.com/visitorManagementV3/integrator/v1/organisations/{organisationId}/meeting/{meetingId}`

Authorization

- Type: `Bearer <ACCESS_TOKEN>`

Request Headers

- `Authorization: Bearer <ACCESS_TOKEN>`

- `Content-Type: application/json`

Path Parameters

| Parameter | Description |

| `organisationId` | The unique identifier of the organization tenant. |

| `meetingId` | The unique identifier of the target meeting to cancel. |

Request Body

None

Field Explanation

None

Postman Verification Steps

1. Open Postman.

2. Create a new request. Set the HTTP method to **DELETE**.

3. Enter the URL: `https://saams.api.spintly.com/visitorManagementV3/integrator/v1/organisations/{organisationId}/meeting/{meetingId}` (replace placeholders).

4. Under the **Headers** tab, add `Authorization: Bearer <ACCESS_TOKEN>` and `Content-Type: application/json`.

5. Click **Send**.

Success Response

json

```
{
  "type": "success",
  "message": "success"
}
```

8. Troubleshooting

400 Bad Request:

- Cause: Payload structure mismatch (e.g. sending a string ID when Spintly expects an integer ID).
- Resolution: Ensure `homeSiteId` and `roles` are mapped as integers, not strings.

- **401 Unauthorized:**

- Cause: The Bearer token has expired or is invalid.
- Resolution: Request a new access token via the `/oauth/token` API.

- **403 Forbidden:**

- Cause: The credentials do not have permission to modify the target organization.

- Resolution: Verify that the `{organisationId}` parameter matches the organization assigned to your API credentials.

9. Database / Integration Notes

To support reliable synchronization, the client application should maintain a mapping between its local user records and the corresponding Spintly user identifiers.

Recommended Database Schema Mapping

Local Column	Mapping Target	Spintly Data Type	Purpose
local_user_id	id	Integer	Links the internal employee ID to the corresponding Spintly user identifier.
assigned_site	homeSiteId	Integer	Links the user's primary site location.
assigned_role	roles	Array of Integers	Links the user's access role.
sync_status	—	String	Tracks the synchronization state (Synced, Pending, Failed).
last_sync_timestamp	—	DateTime	Records the timestamp of the last successful synchronization.

Example Mapping State

A typical user synchronization flow can maintain the following mapping:

- **Local Database User ID:** 1
- **Onboarding Event:** The application creates the user in Spintly.
- **Response Identifier:** Spintly returns the user ID 650126.
- **Database Action:** Store the mapping 1 → 650126.
- **Future Operations:** Use the mapped Spintly user ID for subsequent user-management operations such as updates or deletion.

Critical Deletion Order Rule

When offboarding an employee, the following order should be followed:

1. **Delete the user in Spintly** using the /users/delete endpoint and the mapped Spintly user ID.
2. **Wait for confirmation** that the Spintly deletion request was successful.

3. **Delete the corresponding record** from the local database.

This sequence helps prevent a local database record from being removed while the corresponding Spintly user account remains active.

Onboarding and Synchronization Workflow

A typical user onboarding sequence involves the following steps:

1. **Employee Added to HRMS**
   A new employee record is created in the organization's HRMS or internal system.
2. **Obtain an OAuth Access Token**
   Obtain an access token using the documented OAuth authentication process.
3. **Fetch Required Configuration**
   Retrieve the required configuration information, such as sites, roles, and access points.
4. **Create User in Spintly**
   Create the user using the documented Create User API.
5. **Save the Spintly User ID**
   Store the returned Spintly user ID in the local database mapping.
6. **Assign Access Point Permissions**
   Use the appropriate access-point permissions operation with the required permissionsToAdd payload.
7. **Complete Smart Access Application Setup**
   Complete the user's Smart Access application setup where applicable.
8. **User Logs In and Uses Access**
   The user logs in and uses the access permissions assigned to their account.

Verification Procedures

Developers can verify the results of API operations using the Spintly Dashboard.

User Verification

Log in to the Spintly Dashboard and open the **User Management** section to verify that the user profile has been created or updated correctly.

Access Verification

Inspect the **Access History** section to verify access events associated with the user's assigned access permissions.

Integration Notes

- Maintain the mapping between local user IDs and Spintly user IDs in the client application's database.
- Use the Spintly user ID returned by the API for subsequent user-management operations.
- Perform the Spintly deletion operation before removing the corresponding local database record.

- Retrieve current site, role, and access-point identifiers from the appropriate APIs instead of assuming identifier values.
- Use the appropriate permissions operation depending on whether access needs to be added or removed.
- Keep synchronization status and the last successful synchronization timestamp in the local database to support monitoring and troubleshooting